

数据库默写题（出自往年题）

Week1 & Week2

数据库名词解释

- **Data**
facts and statistics collected together for reference or analysis
- **Database**
a shared collection of logically related data and a description of this data, designed to meet the information needs of an organization
- **DBMS**
a software system that enables users to define, create, maintain, and control access to the database
- **Database application program**
a computer program that interacts with database
- **Data model**
a graphical description of the components of database
- **Data schema**
the description of the database

Relational model 名词解释

- **entity**
a distinct object in the organization that is to be represented in the database
- **relation**
a table with columns and rows
- **relationship**
an association between entities
- **attribute**
a property that describes some aspect of the object that we wish to record
- **domain**
the set of allowable values for one or more attributes
- tuple**
row of a relation
- degree**
the number of attributes in a relation
- cardinality**
the number of tuples in a relation
- **relational database**
a collection of normalized relations with distinct relation names.
- **relational model**
all data is logically structured within relations, contains a set of tables
- **relational algebra**
formal language associated with the relational Model
- **relational schema**
can be thought as the basic information describing a table or relation

key

- 1. 什么是 **candidate key**

a set of attributes that uniquely identifies a tuple within a relation

例子: Student(studentId, studentName, email), studentId is a CK

- 2. 什么是 **primary key**

candidate key selected to identify tuples uniquely with relation

(从 candidate key 里面随便挑了一个或者多个)

Set of attribute(s) that uniquely identifies each row of a relation

注意是 row, 不是 column

- 3. 什么是 **foreign key**

attributes within one relation that matches candidate key of some other relation

Integrity

- 1. 什么是 **entity integrity**

$PK \neq NULL$

in a base relation, no attribute of a primary key can be NULL

- 2. 什么是 **referential integrity**

$FK = CK / NULL$

if foreign key exists:

either foreign key value matches a candidate key value of some tuple in its relation

or foreign key value is NULL

SQL

- 1. 什么是 **DDL** 和 **DML**

DDL(data definition): define database structure, 包括 create/drop table

DML(data manipulation): retrieve and update data, 包括 insert/delete/update/select

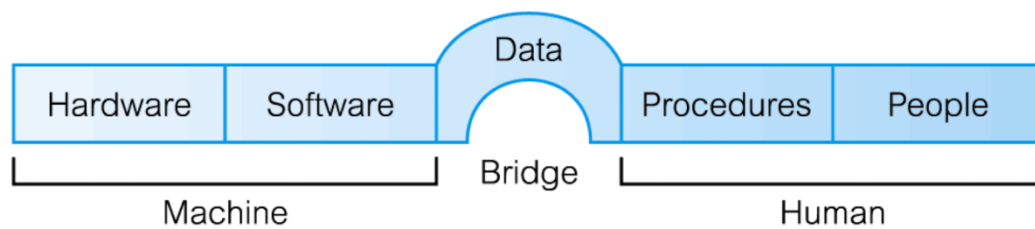
Database 相关内容

DB、DBS、DBMS

1. What are the different roles in Database Environment

Data Administrator (DA)	管理数据的
Database Administrator (DBA)	管理数据库的
Database Designer (Logical and Physical)	设计数据库的
Application Developer	开发程序的
End User (naive and sophisticated)	用户

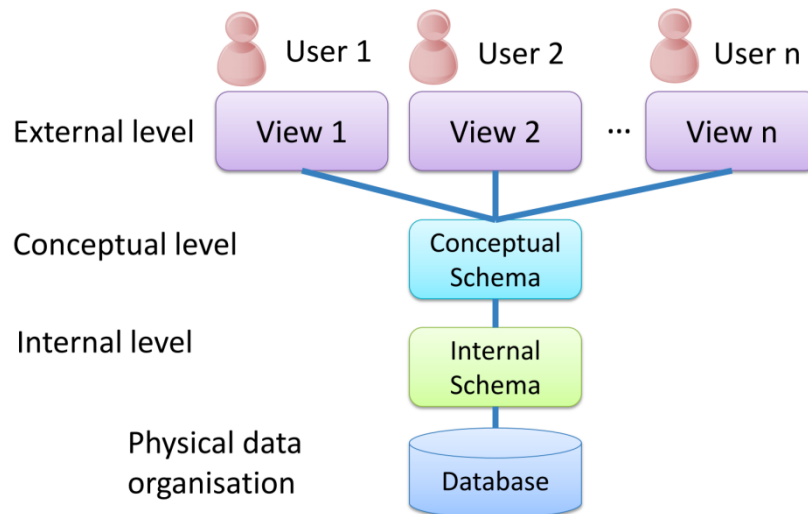
2. Database Environment 的 component



3. 使用 DBS 的几个优点（考过列举 4 个）（这种列举完要解释一下）(ppt 无)

- control of data redundancy
- data consistency
- more information from the same amount of data
- sharing of data
- improved data integrity
- improved security
- enforcement of standards

2. Three level ANSI-SPARC architecture 长什么样子（考过画图）



3. Three level ANSI-SPARC architecture 有哪三层

external level + conceptual level + internal level



ANSI-SPARC	Data abstraction	模型
external level	view level	
conceptual level	logical level	ER model
internal level	physical level	Relational model
physical data organisation		

Design (handwritten note with an arrow pointing from the conceptual level to the internal level)

4. Discuss the reasons/goals for the three-level architecture

separate each user's view of the database from the way the database is physically represented

Database design

1. Database Design Methodology 的 main phases?

the general steps for Database Design Methodology?

Briefly explain the four steps to design a relational database?

- (1) gather requirements
- (2) conceptual database design
- (3) logical database design
- (4) physical database design

2. 什么是 logical database design?

怎么从 ER model 变成 Relational model?

maps the conceptual data model on to a logical model

independent of a particular DBMS and other physical considerations

3. Discuss the relationship between ER model and Relational model

ER model 是 conceptual level 的

Relational model 是 logic level 的

通过 logical database design 的过程可以

map the conceptual data model on to a logical model

independent of a particular DBMS and other physical considerations

4. explain how 1:1 relationship in ER model can be mapped into relations in relational model

place foreign keys in one or both relations

5. explain how 1:m relationship in ER model can be mapped into relations in relational model

post a copy of the primary key attribute of one-side entity into the relation representing the many-side to act as a foreign key

DDBMS

- **1. Distributed database**

a logically interrelated collection of shared data and a description of this data physically spread over a computer network

- **2. Distributed DBMS**

a software system that permits the management of the distributed database and makes the distribution transparent to users.

consists of a single logical database that is split into fragments.

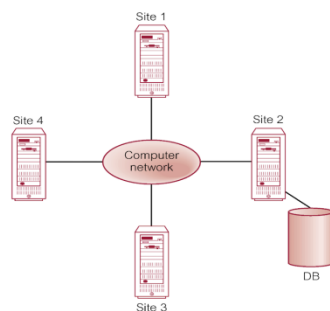
These fragments are distributed over a computer network.

- **3. Distributed processing**

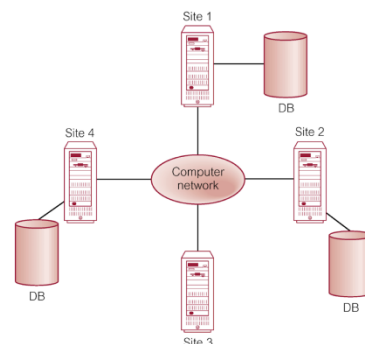
there is a centralised database which can be accessed over a computer network.

4. Is Distributed DBMS the same as distributed processing? Explain.

Distributed Processing



Distributed DBMS



Distributed Processing:

centralized database

can be accessed over a computer network

Distributed DBMS

single logical database that is split into fragments

fragments are distributed over a computer network

5. Which added steps are required in designing DDBMS as compared to designing a normal Relational DBMS?

就是 DDB 设计时的三个 key issues

Fragmentation

horizontal / vertical

Relation may be divided into a number of sub-relations, which are then distributed.

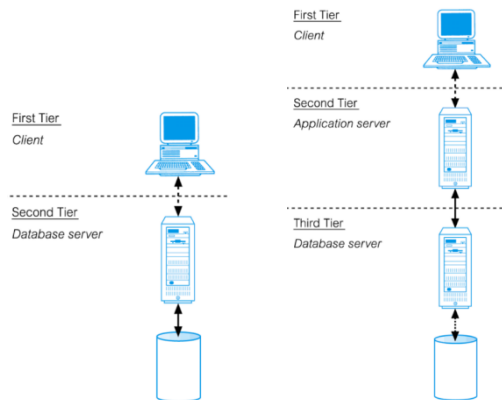
Allocation

Each fragment is stored at the site with “optimal” distribution

Replication

copy of fragment may be maintained at several sites

6. Describe and illustrate the elements of a 3-tier client/server architecture.



	2-tier 的任务	3-tier 的任务
Client	user interface business logic data processing logic	user interface
Application server		business logic data processing logic
Database server	server-side validation database access	data validation database access

7. State ONE reason to use a distributed DBMS instead of distributed processing.

- (1) each site needs to have control over its own data.
- (2) different sites have their own existing DBMS

8. Indicate ONE example of functionality that a distributed DBMS should be able to provide.

- (1) Extended communication services
- (2) Extended Data Dictionary.
- (3) Distributed query processing.
- (4) Extended concurrency control
- (5) Extended recovery services

Week3

Dependency

1. What is Functional dependency? Example?

Functional dependency describes relationship between attributes.

A and B are attributes of relation R, if each value of A in R is associated with exactly one value of B in R, then B is functionally dependent on A.

denoted $A \rightarrow B$

例子: studentId $\rightarrow$ studentName, email

2. What is Transitive dependency? Example?

describes a condition where A, B, and C are attributes of a relation such that if $A \rightarrow B$ and $B \rightarrow C$, then C is transitively dependent on A via B

(provided that A is not functionally dependent on B or C)

例子: Student(studentId, moduleId, moduleName)

studentId $\rightarrow$ moduleId

moduleId $\rightarrow$ moduleName

then, moduleName is transitively dependent on studentId via moduleId

3. What is Multi-valued dependency? Example?

dependency between attributes (for example, A, B, and C) in a relation, such that for each value of A there is a set of values for B and a set of values for C.

However, the set of values for B and C are independent of each other.

Denoted $A \twoheadrightarrow B$ $A \twoheadrightarrow C$

例子: student(studentId, subjects, hobbies)

studentId $\twoheadrightarrow$ subjects

studentId $\twoheadrightarrow$ hobbies

4. What is non-trivial FD? Take $A \twoheadrightarrow B$ for example.

a MVD $A \twoheadrightarrow B$ in relation R is defined as being non-trivial if

(a) B isn't subset of A

(b) $A \cup B \neq R$

a non-trivial FD specifies a constraint

1. the potential problems associated with redundant data

- Insertion anomaly : 违 entity integrity
- deletion anomaly : 删 A, B 丢失
- modification anomaly 改好多行

2. full FD

if A and B are attributes of a relation, B is fully functionally dependent on A, if B is functionally dependent on A, but not on any proper subset of A.

3. partial FD

A FD $A \rightarrow B$ is a partial dependency if there is some attribute that can be removed from A and the dependency still holds.

范式

UNF

- A table that contains one or more repeating groups.
- To create an unnormalized table
 - Transform the data from the information source (e.g. form) into table format with columns and rows.

1NF

- A relation in which the intersection of each row and column contains one and only one value.

2NF

- Based on the concept of full functional dependency.
- Full functional dependency indicates that if
 - A and B are attributes of a relation,
 - B is fully dependent on A if B is functionally dependent on A but not on any proper subset of A.

- 2NF: A relation that is in 1NF and every non-primary-key attribute is fully functionally dependent on the primary key.

3NF

- 3NF: A relation that is in 1NF and 2NF and in which no non-primary-key attribute is transitively dependent on the primary key.

BCNF

- Boyce-Codd normal form (BCNF)
 - A relation is in BCNF if and only if every determinant is a candidate key.

- Nominate an attribute or group of attributes to act as the key for the unnormalized table.
- Identify the repeating group(s) in the unnormalized table which repeats for the key attribute(s).
- Remove the repeating group by
 - Entering appropriate data into the empty columns of rows containing the repeating data ('flattening' the table).Or by
 - Placing the repeating data along with a copy of the original key attribute(s) into a separate relation.

1. Identify the functional dependencies in the relation.
2. Identify the primary key for the 1NF relation.
3. If partial dependencies (on primary key) exist, remove the partially dependent attributes from the relation by placing the attributes in a new relation along with a copy of their determinant.

1. Identify functional dependencies in the relation.
2. Identify the primary key in the 2NF relation.

3. If transitive dependencies (on primary key) exist, remove the transitively dependent attributes from the relation by placing the attributes in a new relation along with a copy of their determinant.

4NF

- 4NF: a relation is in 4NF if and only if for every nontrivial multi-valued dependency $A \twoheadrightarrow B$, A is a candidate key of the relation.

事务

1. What is a transaction?

Action carried out by user or application, which reads or updates contents of database

2. What are the properties ACID of transactions?

Atomicity all transactions are completed or non is completed

Consistency must transform database from one valid state to another

Isolation partial effects of incomplete transactions should not be visible to other transactions

Durability effects of a committed transaction are permanent and mustn't be lost because of later failure

3. Explain the principles of two-phase locking(2PL).

Every transaction must lock an item(read or write) before accessing it.

Once a lock has been released, no new items can be locked.

4. Describe how 2PL works

(1) To read a resource a transaction needs to have a shared lock on that resource. A transaction cannot acquire a shared lock on any resource that has an exclusive lock on it by another transaction.

(2) To write a resource a transaction needs an exclusive lock on it. A transaction can't acquire an exclusive lock if any other transaction has any lock on that resource.

(3) Growing phase: A transaction needs to acquire all the necessary locks and waits if the locks are not available.

(4) Shrinking phase: A transaction only releases the locks when no new locks are needed.

5. What is a deadlock

occurs when 2 transactions are each waiting for locks held by the other to be released

6. Name and explain two techniques that can be used to solve deadlock problems.

Timeouts:

Transaction that requests lock will only wait for a system-defined period of time if lock has not been granted within this period, lock request times out

Deadlock prevention:

DBMS looks ahead to see if the transaction would cause a deadlock and never allows a deadlock to occur

3.3 事务 Concurrency Control

lost update
uncommitted dependency
inconsistent recovery

1. serialisability

- (1) Objective of a concurrency control protocol is to schedule transactions to avoid any interference.
- (2) Could run transactions serially, but this limits degree of concurrency or parallelism in system.
- (3) Serialisability identifies those executions of transactions guaranteed to ensure consistency.
- (4) Objective of serialisability is to find nonserial schedules that allow transactions to execute concurrently without interfering with one another.
- (5) The objective of serialisability is to find nonserial schedules that are equivalent to some serial schedule. Such a schedule is called serialisable.

2. Locking

- (1) Transaction uses locks to deny access to other transactions and so prevent incorrect updates.
- (2) A transaction must claim a read or write lock on a data item before read or write.
- (3)

Read-lock allows several transactions simultaneously to read a resource (but no transactions can change it at the same time)

Write-lock allows one transaction exclusive access to write to a resource. No other transaction can read this resource at the same time.

(4)

Before **reading** from a resource a transaction must acquire a **read-lock**

Before **writing** to a resource a transaction must acquire a **write-lock**

(5) Locks are released on commit/rollback

(6) If the requested lock is not available, transaction waits

3. 2PL

定义 All locking operations precede unlock operation in the transaction.

Principle 考过

两个阶段 growing + shrinking

4. Timestamping

- (1) Transactions are ordered globally so that older transactions (i.e., with smaller timestamps) get priority in the event of conflict.
- (2) Read/write proceeds only if last update on that data item was carried out by an older transaction.
- (3) Otherwise, transaction requesting read/write is restarted and given a new timestamp.

2. transaction log file 的目的

keep track of current state of transactions and database changes

Contains information about all updates to database:

Transaction records + Checkpoint records.

3. checkpoint 的目的

checkpoint: point of synchronization between database and log file.

enables updates to database in progress to be made permanent.

4.2 Data Security + Ethics

1. 什么是 database security

Mechanisms that protect the database against intentional or accidental threats.

2. 数据库安全的 6 个方法

(1) Authorization

The process of authorization involves authentication of subjects requesting access to objects.

Authentication is a mechanism that determines whether a user is, who he or she claims to be.

(2) Access control

Based on the granting and revoking of privileges.

(3) Views

(1) A view is a virtual relation that does not actually exist in the database.

(2) Views are queries that are saved in the database and generally read-only

(3) The view provides a security mechanism by hiding parts of the database from certain users.

(4) Backup and recovery

Backup

process of periodically taking a copy of the database and log file to offline storage media.

Recovery

process of restoring database to a correct state in the event of a failure.

(5) Integrity

Prevents data from becoming invalid, and hence giving misleading or incorrect results.

(6) Encryption

the encoding of the data by a special algorithm that renders the data unreadable by any program without the decryption key.

3. SQL 注入攻击是什么

a web security attack that allows an attacker to interfere with the queries that an application makes to its database.

登录



cookie



浏览



备份/恢复



integrity



加密

4. 怎么防止 SQL 注入攻击

- Using prepared statements (parameterized statements)
 - Instead of concatenating user input in the statement, use a placeholder(parameter) to take user input.
 - A placeholder can only store a value of the given type; hence SQL injection would be treated as a strange (invalid) parameter value.
- Escaping all user supplied input
 - E.g. a single quote (') in an input string should be prepended with a backslash (\) so that the database understands the single quote is part of a given string, rather than its terminator.
- Pattern check
 - Parameters can be checked to determine if the value is a valid representation of the given type. Strings can be checked to adhere to a specific pattern (e.g. dates, phone numbers)
- Database permissions
 - Limiting the permissions on the database login used by the web application to only what is needed
- Detecting SQL injection vulnerabilities, manually or using web vulnerability scanner.

5. 什么是 data ethics

a branch of ethics that evaluates data practices that have the potential to adversely impact people and society.

involves the moral obligations of gathering, protecting, and using data, especially personally identifiable information (PII) and how it affects individuals.

Ethics

TO PA

1. Explain the 4 principles in Data Ethics.

Ownership

an individual has ownership over their personal information.

Transparency

clear communication of what data will be gathered, how it will be stored or used, and who it might be shared with.

Data Privacy

ensure data subjects' privacy

Accountability

an organization takes responsibility for what happens to an individual's data after collecting it.

XML

1. What is XML *extensible markup language*

a meta-language that enables designers to create their own customised tags to provide functionality not available with HTML.

(meta-language: a language for describing other languages)

2. Explain the advantages of using XSD over DTD

(1) XSD is more comprehensive method of defining content model of an XML document.

(2) XSD tries to overcome XML deficiencies by using additional expressiveness to allow Web applications that exchange XML data more robustly.

3. Discuss the difference between XML and the relational model in terms of structure, schema, queries, ordering and implementation.

	Relational	XML (Semi-Structured data)
Structure	Tables, columns, rows	Hierarchical, tree
Schema	Fixed in advance	Self-describing, flexible
Queries	SQL, simple language, standard	Not so simple
Ordering	None	Ordered
Implementation	Mature, native	Add-on

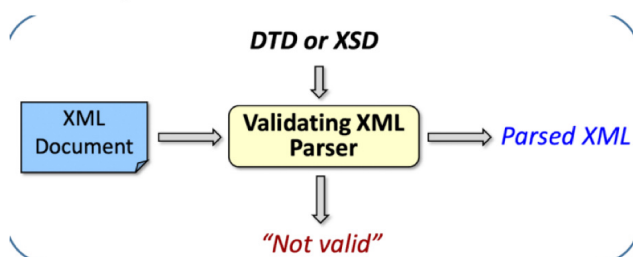
	Relational	XML
结构	使用表格、行列	采用层次结构或树形结构
模式	提前固定 数据表的结构需要在存储数据之前定义	自描述的、灵活 可以自主描述数据结构, 灵活应对不同的数据格式
查询	使用 SQL 语言, 简单且标准化	不简单
排序	没有	有
实现	成熟并且原生支持	通常作为附加功能

2. 什么是 well-formed/valid XML

(1) Single root element

(2) Matched tags, proper nesting

(3) Unique attributes within elements



3. DTD 的目的是什么

Defines the valid syntax of an XML document. (element+attribute)

NoSQL

1. Explain NoSQL

Not only SQL 或者是

Not Relational SQL

NoSQL is an alternative, non-traditional DB technology to be used in large scale environments where (ACID) transactions are not a priority.

2. NoSQL 的 key features

flexible schema

quicker + cheaper to set up

massive scalability

relaxed consistency -> higher performance + availability

3. 用 relational database 而不用 NoSQL 这种 semi-structured 的例子

transaction database: Visa or Amazon

They need structured data with transaction semantics.

Inventory records

Financial records

They all require frequent updates + reads

require high integrity and atomicity (ACID properties)

4. 用 NoSQL database 而不用 relational database 的例子

Instagram

(1) Too many data

Instagram has a large number of users that generate enormous images and contents all the time.

(2) Gain performance and scalability

Instagram requires storing large amount of data that need fast scalability while maintaining high performance.

(3) no ACID constraints

There is no strict ACID constraints on a social media platform such as Instagram.

5. 用 NoSQL 这种 semi-structured 比 relational Database 坏在哪里?

no declarative query language -> more programming

relaxed consistency -> fewer guarantees

6. semi-structured data storage 比 relational database 好在哪里?

(1) Semi-structured or flat files based data stores are best for **massive data** that is read frequently, but with minimal updates.

(2) There is much **less overhead** to process data in this format.

(3) We also have the **flexibility** to process data that doesn't have a completely fixed structure.

4.4 NoSQL

1. Distributed management 的三个性质: CAP

1. Consistency → A data item has the same value at the same time (to ensure coherency).
2. Availability → Data is available, even if a server is down.
3. Partition Tolerance → A query must have an answer, even if the system is partitioned (unless there is a global failure).

2. NoSQL 的 application area

NoSQL is an alternative, non-traditional DB technology to be used in large scale environments where (ACID) transactions are not a priority.

注意:

WAIT 都是大写

commit/rollback 那里要记得写上 unlock(X)

read: read_lock
write: write_lock
unlock(X,Y,Z)

lost update problem
uncommitted dependency problem
inconsistent analysis
dead lock
cascading roll back

总结:

checkpoint 前: changes written to secondary storage

checkpoint 后 failure 前: redo

failure 后: undo, 都 failed 了, 肯定要取消啊

<?xml version="1.0" encoding="UTF-8"?>

<!-- 这是一本书的XML示例 -->

<!DOCTYPE book [

<!ENTITY authorName "J.K. Rowling">

<book category="fantasy" isbn="978-3-16-148410-0">

<title>哈利 & 波特</title>

<author>&authorName;</author>

<summary>魔法世界的冒险故事.</summary>

</book>

<?xml version="1.0"?>

<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">

<xs:element name="person">

<xs:complexType>

<xs:sequence>

<xs:element name="name" type="xs:string"/>

<xs:element name="age" type="xs:integer"/>

</xs:sequence>

<xs:attribute name="id" type="xs:string" use="required"/>

</xs:complexType>

</xs:element>

</xs:schema>

分布式

horizontal fragmentation → $\sigma_{dNo = "001"}(Employee)$ $\sigma_{dNo = "002"}(Employee)$

vertical fragmentation → $\pi_{dNo, dName}(Department)$ $\pi_{dNo, mgr_eld}(Department)$

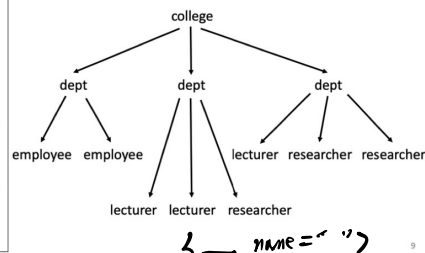
(Solution) – Total transfer costs $\sum(\text{条数} \times \text{字节数})$

Strategy 1 → $5500 \times 15 + 100 \times 30 = 85500$ bytes (most economical)

Strategy 2 → $5500 \times 15 + 5500 \times 8 = 126500$ bytes

XML

```
<college>
  <dept name="Admin">
    <employee>John</employee>
    <employee>Dina</employee>
  </dept>
  <dept name="Engineering">
    <lecturer>Alma</lecturer>
    <lecturer>James</lecturer>
    <researcher>David</researcher>
  </dept>
  <dept name="Computing">
    <lecturer>Alina</lecturer>
    <researcher>James</researcher>
    <researcher>Alma</researcher>
  </dept>
</college>
```



(Solution)

Department:

name
Admin
Engineering
Computing

Employee: $\frac{1}{1} \rightarrow$

dept	employee
Admin	John
Admin	Dina

Lecturer:

dept	lecturer
Engineering	Alma
Engineering	James
Computing	Alina

Researcher:

dept	researcher
Engineering	David
Computing	James
Computing	Alma

4. XSD与DTD的区别

- DTD: 早期的XML模式, 语法不基于XML, 能力有限, 类型简单。
- XSD: 基于XML自身, 功能更强大, 可自定义类型, 支持数据类型, 内容可扩展、易于验证、国际标准。